# SignalLab

ITT 036 Programming Assignment 1 | One-page specification | 20 October 2026

## Objective and implementation

A small C++17 command-line digital signal encoder and decoder. The application uses only the C++ standard library (including random, vector, cmath, fstream and filesystem). Existing team modules are retained. SVG graphs are written directly in C++; no graphics framework or runtime Python dependency is required.

## Features and workflow

Select digital or analog input. Digital input accepts typed bits or random streams with optional guaranteed four/eight-zero runs. Expand-around-center search reports the longest contiguous palindrome, zero-based position and length; ties choose the earliest ($O(n^2)$ time, O(1) working space). Choose NRZ-L, NRZ-I, Manchester, Differential Manchester or AMI; AMI offers B8ZS/HDB3 scrambling. After graph export, optional level-based decoding displays the recovered bits and a round-trip comparison. Analog input goes through PCM encoding and decoding and can feed any line encoder.

## Signal conventions

Normalized levels: +1 V, 0 V, -1 V. NRZ-L: 0=+1, 1=-1. NRZ-I: initial +1, toggle on 1. Manchester: 0=(+1,-1), 1=(-1,+1), two half-bit levels. Differential Manchester: initial +1, transition at every midpoint and at the bit boundary for 0. AMI: first ordinary mark +1, subsequent marks alternate; virtual previous mark -1. B8ZS substitutes 000VB0VB. HDB3 substitutes 000V after odd ordinary-mark parity since the last substitution, otherwise B00V; reset parity after substitution. V repeats the preceding nonzero polarity and B alternates it. Encoding/scrambling and decoding take O(n) time.

## PCM sampling and reconstruction

Manual samples are assumed uniformly sampled at the entered Fs (1-1000000 Hz); the generated sine uses frequency Fs/8, midpoint offset and half-range amplitude. For b=1-8 bits, use $L=2^b$ uniform intervals and step D=(maximum-minimum)/L. Encode the interval index MSB first; the maximum endpoint uses the last interval. Decode to interval midpoints and hold each value for 1/Fs seconds (zero-order hold). Reject nonfinite/out-of-range samples. Example: range [-1,1], b=2, samples [-1,0,1] give 00 10 11 and [-0.75,0.25,0.75].

## Build, run and output

From the repository root, compile the following as one command; run `./signallab` (Windows: `.\signallab.exe`).

```
g++ -std=c++17 main.cpp data_generator.cpp palindrome.cpp line_coding.cpp pcm.cpp waveform.cpp -o signallab
```

Open `output/line_waveform.svg` or `output/pcm_staircase.svg` in a browser. Graphs show at most 64 bits/samples; matching CSVs retain the full data. Scrambling graphs include ordinary AMI for comparison. New runs overwrite matching output files.

## Verification and limitations

C++17 compilation, exact voltage/parity examples, seven decoder round trips, generator and palindrome cases, PCM quantization and timed exports passed focused checks; 26 CLI runs and SVG parsing also passed. Input limits: 10000 bits and 1000 PCM samples. Decoding analyzes ideal clock-aligned signal levels, not raster images; the stricter image-analysis extra-credit target is not implemented. No hardware ADC, noise recovery or analog low-pass reconstruction filter is included. PCM is the selected modulation option.

## References

ITT 036 Programming Assignment 1, 21 September 2026, pp. 1-2; existing SignalLab team source: github.com/DeepakPatel004/SignalLab. This PDF was prepared using Python ReportLab (report-generation tooling only).